

阅读建议：先看第 1-4 节理解研究主线，再看 A1/A2 代码讲解，最后背第 13-17 节的汇报口径。

中期研究小白讲透版

这份文档按“完全小白也能听懂”的方式，把中期 PPT 背后的研究从头到尾串起来。

你不要把这个课题理解成“我让大模型帮我写了几个代码”。更准确地说，你做的是：

把大模型参与数学建模 / 数值计算作业的过程，整理成一条可以检查、可以复现、可以评分的证据链。

证据链的形态是：

题目要求

-> Task Card

-> Prompt 模板

-> 模型生成代码

-> 本地运行评测脚本

-> 输出图像 / JSON

-> run 文档

-> metrics CSV

-> 阶段总结

-> 中期 PPT

COE

Reader：读题，说明输入输出和验收条件

Planner：规划算法步骤和风险点

Coder：写代码和关键断言

Reviewer：检查常见错误

Experimenter：说明怎么测试、怎么记录结果

direct_answer：直接要答案

plain_guidance：给任务卡，让它按清单答

coe_guided：给任务卡，还强制分角色一步步答

中期之前最重要的结论不是“结构化提示一定让最终结果更好”，而是：

1. A1 / A2 两个主案例已经闭环。2 means：给模型更多结构化提示后，它不一定让最终结果分数更高，但它更容易把“我是怎么检查的、错在哪里、修完怎么验证”讲清楚。这样老师或助教更容易判断学生是不是真的理解了过程。
2. plain_guidance / coe_guided 更稳定地产生自检、根因解释和回归说明。
3. A2 的最终图像恢复指标三组提示完全一致，所以不能夸大成“结构化提示提高图像质量”。
4. 正式受试者还没采集，中期只能说研究链路和预试验材料已经准备好。

1. 你是怎么开始的

一开始你的题目是“大模型在数学建模与求解教学中的应用方法研究”。这个题很大，如果直接做，会变成泛泛地比较“哪个回答看起来更好”。

所以你先做了一个关键收束：

宽泛题目：大模型辅助数学建模教学

- > 可落地问题：大模型如何参与数值 / 编程类作业
- > 可评估形式：生成代码、运行代码、记录输出、回填指标
- > 中期主案例：A1 Seam Carving + A2 RSLT 图像修复

为什么选这两个案例？

因为它们同时满足五个条件：

1. 可以在本地跑，不依赖外部接口。
2. 可以评分，比如正确性、PSNR、SSIM、RSE。
3. 可以人为设计固定 bug。
4. 可以在三种 prompt 条件下做对照。
5. 可以压缩成 20 到 30 分钟的试用任务。

A3 曲线拟合和 A4 Poisson 融合已经有入口，但中期不拿它们支撑主结论，因为它们还没有进入完整实验链路。

2. 整体研究逻辑

你真正研究的不是某个算法本身，而是“LLM 参与算法作业时，怎样把过程变得可检查”。

完整逻辑可以画成这样：

学生或研究者拿到题目

- > 先把题目整理成 Task Card
- > 再把 Task Card 放进不同 prompt 模式
- > 让模型生成代码
- > 用统一评测协议运行代码
- > 保存代码、图像、JSON、运行记录
- > 把结果写进 CSV
- > 比较不同提示方式的过程质量和结果质量

这里的“过程质量”包括：

- 有没有明确输入输出。
- 有没有说清约束和不变量。
- 有没有显式自检。
- 出错时有没有说明根因。
- 修复后有没有回归验证。

这里的“结果质量”包括：

- 代码能不能运行。
- 输出尺寸和语义对不对。

- 图像指标如何。
- 是否生成了要求的文件。

所以你一直在区分两件事：

结果正确
过程可解释

中期最稳的发现就是：很多时候最终结果都能正确，但结构化指导让过程痕迹更完整。

3. 仓库每个主要目录是做什么的

你可以这样理解仓库：

目录	作用	汇报时怎么说
problems/	原始任务要求	题目从这里进入研究链路
task_cards/	把题目结构化后的任务卡	把目标、输入输出、算法步骤、常见错误写清楚
prompts/	三种提示模板	direct / plain / CoE 的对照条件
solutions/	模型生成的代码、prompt、raw response	模型真正产出的 artifact
outputs/	代码运行后的图像和 JSON	结果证据
runs/	每一次实验的文字记录	记录 run_id、模式、结果、问题
metrics/	CSV 指标表	用来做横向比较
report/	协议、总结、PPT、讲稿	中期材料和阶段结论
pilot/	预试验表单和任务包	后续正式受试者采集入口
hw2-op2/	A2 原始项目本体	低秩图像修复的 gold 参考

这些目录不是随便放文件，而是围绕同一条证据链服务。

4. 三种 prompt 条件

你比较的是三种提示方式：

4.1 direct_answer

最弱结构。只给题目和接口，直接要求模型给完整代码。

A1 例子：

输入给模型：

- 1. 作业是 Seam Carving
- 2. 主函数是 seam_carve_image(im, sz)
- 3. 要输出可运行代码和对比图
- 4. 只做 shrink，不做 enlarge

它的作用是当基线：如果不提供 Task Card 和检查清单，模型能做到什么程度。

4.2 plain_guidance

中等结构。给题目，也给 Task Card 和评测协议，但不要求多角色分工。

它会告诉模型：

你要按 Task Card 检查输入输出、约束、不变量、实验输出。

这个模式的特点是：不会太复杂，但已经开始强制模型讲过程。

4.3 coe_guided

最高结构。它把一次求解拆成五个角色：

角色	负责什么
Reader	重述题目、输入输出、验收条件
Planner	算法步骤、函数拆分、边界风险
Coder	代码骨架、关键实现、自检断言
Reviewer	对照常见错误逐项审查
Experimenter	固定测试、输出、记录和指标

这个模式的目的是不是让答案更长，而是让“为什么这么做”和“怎么检查”更清楚。

5. A1 主案例：Seam Carving 是什么

A1 的任务是内容感知图像缩放。

普通 resize 会把整张图按比例压缩，主体也会变形。普通 crop 会直接裁掉边缘，可能丢信息。Seam Carving 的想法是：

找出一条图像里“不重要”的路径 seam
-> 删除它
-> 图像宽度减少 1
-> 重复删除，直到达到目标宽度

5.1 A1 输入是什么

主函数是：

```
seam_carve_image(im, sz)
```

输入：

- `im`：原图，形状是 `H x W x C`。
- `sz`：目标尺寸，写成 `(target_h, target_w)`。

中期固定协议里使用两张图：

- `solutions/hw1_op1/figs/bing1.png`
- `solutions/hw1_op1/figs/original.png`

评测脚本会把图片缩到最长边不超过 420，方便运行。

5.2 A1 输出是什么

每个 case 会输出：

输出文件	含义
<code>original.png</code>	原图
<code>resize_linear.png</code>	普通线性 resize 结果
<code>crop.png</code>	中心裁剪结果
<code>seam_width.png</code>	只缩宽的 seam carving 结果
<code>seam_height.png</code>	只缩高的 seam carving 结果
<code>compare_grid.png</code>	多图对比网格

例如 run 007 的输出目录是：

```
outputs/hw1_op1/run_007_plain_guidance/
```

5.3 A1 关键代码文件

代表性代码是：

```
solutions/hw1_op1/generated/run_007_plain_guidance/src/seam_carving_generated.py
solutions/hw1_op1/generated/run_007_plain_guidance/src/run_protocol_eval.py
```

第一个文件是真正的算法实现，第二个文件是评测脚本。

5.4 seam_carving_generated.py 每个函数做什么

LAPLACIAN_KERNEL

```
LAPLACIAN_KERNEL = np.array([
    [0.5, 1.0, 0.5],
    [1.0, -6.0, 1.0],
    [0.5, 1.0, 0.5],
])
```

这是一个边缘检测核。它会让图像中变化剧烈的位置能量更高。

直觉:

边缘 / 人物 / 文字 / 纹理 -> 能量高 -> 不适合删
平坦背景 -> 能量低 -> 更适合删

compute_energy(im)

作用: 计算每个像素的重要性。

流程:

1. 把图像转成浮点数。
2. 对每个颜色通道做卷积。
3. 把每个通道的边缘强度平方后相加。
4. 得到二维能量图 E 。

输入:

im: H x W x C

输出:

energy: H x W

find_vertical_seam(energy)

这是 A1 最关键的函数。

目标: 从上到下找一条累计能量最小的路径。

核心变量:

变量	含义
dp	到达每个像素位置的最小累计能量
parent	记录当前像素上一行来自哪一列
seam	最后回溯出的路径

关键代码逻辑：

```
left = max(j - 1, 0)
right = min(j + 1, w - 1)
prev = dp[i - 1, left:right + 1]
k = int(np.argmin(prev))
parent[i, j] = left + k
dp[i, j] = energy[i, j] + prev[k]
```

这段是在处理动态规划的边界。

每个像素只能从上一行的三种位置过来：

```
左上 j-1
正上 j
右上 j+1
```

但如果当前像素在第一列，就不能访问 `j-1`；如果在最后一列，就不能访问 `j+1`。所以用 `max` 和 `min` 防止越界。

回溯部分：

```
seam[h - 1] = int(np.argmin(dp[h - 1]))
for i in range(h - 2, -1, -1):
    seam[i] = parent[i + 1, seam[i + 1]]
```

意思是：

1. 先在最后一行找到累计能量最小的位置。
2. 再沿着 `parent` 一路往上找。
3. 得到整条 `seam`。

自检部分：

```
assert np.all(np.abs(np.diff(seam)) <= 1)
assert np.all((seam >= 0) & (seam < w))
```

这两行是结构化指导的重要痕迹：

- 第一行检查 seam 是否连续。
- 第二行检查 seam 是否没有越界。

remove_vertical_seam(im, seam)

作用：把找到的 seam 从图像中删掉。

关键代码：

```
mask = np.ones((h, w), dtype=bool)
mask[np.arange(h), seam] = False
out = im[mask].reshape(h, w - 1, im.shape[2])
```

含义：

1. 先创建一个全 `True` 的布尔 mask。
2. 把 seam 经过的位置标成 `False`。
3. 用 mask 取出剩下的像素。
4. reshape 回 `H x (W-1) x C`。

输出宽度必须减少 1：

```
assert out.shape[1] == w - 1
```

seam_carve_width(im, target_w)

作用：不断删除竖直 seam，直到宽度达到目标宽度。

循环逻辑：

```
while out.shape[1] > target_w:
    energy = compute_energy(out)
    seam = find_vertical_seam(energy)
    out = remove_vertical_seam(out, seam)
```

注意：每删一次 seam 后都会重新计算 energy。

这很重要，因为图像变了，能量图也应该变。

seam_carve_height(im, target_h)

作用：缩小高度。

它没有重新写一套“水平 seam”算法，而是：

```
transposed = np.transpose(im, (1, 0, 2))
carved_t = seam_carve_width(transposed, target_h)
return np.transpose(carved_t, (1, 0, 2))
```


意思是：

把图像转置

- > 高度变成宽度
- > 复用 `seam_carve_width`
- > 再转置回来

这是 A1 的一个关键设计点。

`seam_carve_image(im, sz)`

这是总入口。

它做三件事：

1. 读取目标尺寸。
2. 检查是否只做缩小。
3. 先缩宽，再缩高。

关键边界：

```
if target_h > h or target_w > w:  
    raise ValueError('shrink-only baseline')
```

中期阶段只研究 shrink，不研究 enlarge。

5.5 A1 评测脚本 `run_protocol_eval.py` 做什么

它不是算法本身，而是“把算法放到固定协议下跑一遍”。

流程：

1. 找到仓库根目录。
2. 读取两张测试图。
3. 如果图片太大，缩到最长边 420。
4. 设定目标尺寸：
 - 宽度缩到原来的 80%。
 - 高度缩到原来的 85%。
5. 生成普通 `resize` 和 `crop` 作为对照。
6. 调用 `seam_carve_image` 生成 `seam carving` 结果。
7. 保存所有图像。
8. 保存 `compare_grid.png`。

输入：

```
solutions/hw1_op1/figs/bing1.png
solutions/hw1_op1/figs/original.png
```

输出:

```
outputs/hw1_op1/run_007_plain_guidance/case_subject_bing1/*.png
outputs/hw1_op1/run_007_plain_guidance/case_landscape_original/*.png
```

5.6 A1 指标怎么回填

A1 的主指标表是:

```
metrics/a1_guidance_eval_v0.csv
metrics/a1_codegen_perf_v0.csv
metrics/a1_failure_repair_eval_v0.csv
```

a1_guidance_eval_v0.csv 里每一行是一条 run。

关键列:

列	含义
run_id	本次实验编号
mode	direct / plain / coe
artifact_complete	产物是否完整
runnable	是否能运行
correct	结果是否正确
self_check	自检是否充分
fix_rounds	修了几轮
time_to_first_working_min	第一次跑通用了多久

A1 结果:

- 三种模式最终正确性都达标。
- direct 的 self_check 低。
- plain 和 coe 的 self_check 都满。

所以 A1 支撑的结论是:

```
结构化指导更能留下自检和回归说明。
```

不是：

结构化指导显著提高最终正确率。

6. A2 主案例：RSLT 图像修复是什么

A2 一开始不是直接定成这个任务。早期口径更宽，像“SVD 图像压缩”或低秩图像任务。后来为了可复现、可评分，你把它收束成：

低秩图像任务族下的灰度图像修复主案例

具体函数是：

```
hw2-op2/src/chapter5_rslt.py::rslt_inpainting(...)
```

6.1 A2 要解决什么问题

给你一张被污染或缺失的灰度图：

observed

再给你一个 mask：

mask == 1 表示这个像素是已知的
mask == 0 表示这个像素是缺失 / 被污染的

目标是恢复出：

recovered

也就是尽量接近原图。

6.2 A2 输入是什么

主函数：

```
rslt_inpainting(observed, mask, ...)
```

输入：

输入	含义
<code>observed</code>	退化后的灰度图， <code>H x W</code> ， 范围 <code>[0,1]</code>
<code>mask</code>	同尺寸掩码， 1 是已知， 0 是缺失
<code>patch_size</code>	patch 大小
<code>stride</code>	patch 采样步长
<code>search_window</code>	搜索相似 patch 的窗口
<code>num_similar</code>	找几个相似 patch
<code>rpca_max_iter</code>	RPCA 最大迭代次数
<code>outer_iter</code>	外层恢复迭代次数
<code>true_image</code>	如果有原图， 用来计算 PSNR

6.3 A2 输出是什么

输出两个东西：

```
return recovered, history
```

输出	含义
<code>recovered</code>	修复后的图
<code>history</code>	迭代过程记录， 比如 <code>psnr</code> 、 <code>sparse_energy</code>

评测脚本还会输出：

输出	含义
<code>compare.png</code>	原图 / 退化图 / 修复图对比
<code>summary.json</code>	单个 case 的指标和 history
<code>eval_summary.json</code>	整个 run 的汇总
metrics CSV 行	用于中期统计

6.4 A2 关键代码文件

gold 参考：

```
hw2-op2/src/chapter5_rslt.py
```

代表性生成代码：

```
solutions/hw2_op2/generated/run_032_coe_guided/src/a2_generated.py
solutions/hw2_op2/generated/run_032_coe_guided/src/run_protocol_eval.py
```

6.5 A2 算法直觉

A2 的核心思想是 patch-group RPCA。

先把图像切成很多小块 patch，然后为每个 patch 找相似 patch。相似 patch 堆在一起后，会形成一个矩阵：

```
group_matrix
```

这个矩阵可以分解成：

```
group_matrix = L + S
```

其中：

- **L** 是低秩部分，代表重复纹理和主要结构。
- **S** 是稀疏部分，代表污染、异常、文字、划痕。

恢复时应该写回 **L**，不要写回 **S**。

这正是 A2 里最关键的不变量之一。

6.6 a2_generated.py 每个关键函数做什么

路径设置

```
THIS_DIR = os.path.dirname(os.path.abspath(__file__))
ROOT_DIR = ...
GOLD_SRC = os.path.join(ROOT_DIR, 'hw2-op2', 'src')
```

作用：让生成代码可以导入原项目里的工具函数，比如 `compute_metrics`、`find_similar_patches`、`rpca_ialm`。

```
rslt_patch_rpca(group_matrix, ...)
```

作用：对一组相似 patch 做 RPCA。

流程：

1. 计算 group 均值。
2. 减掉均值，得到 centered matrix。
3. 调用 `rpca_ialm` 做低秩 + 稀疏分解。
4. 把均值加回低秩部分。

输出:

```
low_rank + group_mean  
sparse
```

其中恢复图像用的是 low_rank, 不是 sparse。

`_validate_state(observed, mask_bool, current, weight_map)`

这是 CoE 版本中很重要的自检函数。

它检查:

1. 当前图像 shape 有没有变。
2. weight map shape 有没有变。
3. observed 区域是否仍然和输入一致。

关键断言:

```
assert np.allclose(current[mask_bool], observed[mask_bool])
```

这句的意思是:

mask 标记为已知的像素, 恢复过程中不能被改掉。

`rslt_inpainting(...)`

这是 A2 主函数。

它的流程可以拆成 8 步:

1. 把输入转为浮点数组。
2. 检查 `observed.shape == mask.shape`。
3. 用 `fill_missing_with_mean` 初始化缺失区域。
4. 强制保留 observed 区域:

```
current[mask_bool] = observed[mask_bool]
```

5. 枚举 patch 位置。
6. 对每个 patch 找相似 patch, 堆叠成矩阵。
7. 对矩阵做 RPCA, 只拿低秩部分还原 patch。
8. 聚合所有 patch, 归一化, 然后再次强制施加 mask 约束。

最关键代码：

```
current = accumulator / np.maximum(weight_map, 1e-12)
current[mask_bool] = observed[mask_bool]
current = np.clip(current, 0.0, 1.0)
_validate_state(observed, mask_bool, current, weight_map)
```

逐句解释：

- `accumulator / weight_map`：多个 patch 覆盖同一像素，需要加权平均。
- `np.maximum(weight_map, 1e-12)`：防止除以 0。
- `current[mask_bool] = observed[mask_bool]`：保留已知像素。
- `clip`：保证输出范围在 `[0,1]`。
- `_validate_state`：检查 shape 和 mask 不变量。

6.7 A2 评测脚本做什么

代表文件：

```
solutions/hw2_op2/generated/run_032_coe_guided/src/run_protocol_eval.py
```

它做的事：

1. 读取 `lena` 和 `barbara` 两张固定测试图。
2. 统一转成 256 x 256 灰度图。
3. 生成两种 corruption:
 - `random_pixel@50%`
 - `text@50%`
4. 调用 `apply_mask` 得到 `observed`。
5. 调用 `rslt_inpainting` 得到 `recovered`。
6. 计算 PSNR / SSIM / RSE。
7. 保存 `compare.png`。
8. 保存每个 case 的 `summary.json`。
9. 保存整个 run 的 `eval_summary.json`。

固定协议一共 4 个 case：

```
lena / random_pixel@50%
lena / text@50%
barbara / random_pixel@50%
barbara / text@50%
```

6.8 A2 指标怎么理解

A2 主要指标:

指标	含义
PSNR	越高越好，图像误差越小
SSIM	越接近 1 越好，结构越像原图
RSE	越低越好，相对误差越小
runtime	运行时间
output_ok	是否成功输出

固定协议下的结果:

mode	PSNR	SSIM	RSE	output_ok
direct_answer	25.42	0.756	0.145	20/20
plain_guidance	25.42	0.756	0.145	20/20
coe_guided	25.42	0.756	0.145	20/20

这就是 PPT 里必须主动讲的边界:

A2 最终指标没有拉开差异。

所以正确说法是:

结构化指导改善的是过程质量和可复查性。

不能说:

结构化指导显著提升了 A2 图像恢复质量。

6.9 为什么 A2 三组指标一样

因为 A2 固定协议下:

- 图像固定。
- mask 生成方式固定。
- 参数固定。
- 算法逻辑基本一致。
- 一旦 mask 语义和低秩写回处理正确，输出就接近确定。

所以三组 prompt 主要差在“过程说明”，不是差在最终图像。

这反而让你的结论更诚实：

我的研究不是硬说 AI 提高数值指标，
而是说明结构化流程能让生成和评分更可靠。

7. A2 expanded-scope 是什么

固定协议只有 2 张图和 2 种 corruption。为了检查链路是不是只对这 4 个 case 有效，你做了 expanded-scope。

设置：

维度	取值
图像	lena, barbara, peppers, cameraman
corruption	random_pixel@30%, random_pixel@70%, center_block@35%, text@30%
总 case	$4 \times 4 = 16$
三种 mode	$16 \times 3 = 48$ 条性能记录

结果：

- 48 个 case 全部 output_ok。
- 三种 mode 在每个 case 上的 PSNR / SSIM / RSE 仍然完全一致。
- 说明链路不只在最初 4 个 fixed cases 上稳定。

但注意：expanded-scope 复用已有 baseline 代码产物，不是重新生成代码。

8. Bug repair 是什么

Bug repair 是你故意准备一些有代表性的坏代码，让模型或流程去诊断和修复。

它的意义是：教学里不能只看“从零生成”，还要看学生能不能读懂错误、定位错误、最小修改、回归验证。

8.1 A1 的三个 bug

bug	问题	为什么重要
bug_01_dp_boundary	DP 第一列 / 最后一列边界处理错	seam 会越界或累计代价错
bug_02_no_energy_recompute	删除 seam 后不重新计算 energy	后续 seam 基于旧图，逻辑错
bug_03_height_transpose	高度缩小时转置复用错	输出尺寸或通道语义错

8.2 A2 的三个 bug

bug	问题	为什么重要
bug_01_missing_mask_constraint	没有重新施加 mask 约束	已知像素被改掉
bug_02_mask_polarity_inverted	mask 1/0 语义反了	把已知和缺失搞反
bug_03_sparse_component_written_back	写回了 S 而不是 L	把污染当成恢复结果

8.3 bug repair 的指标

指标	含义
diagnosis_correct	根因说对了吗
patch_runnable	修补后的代码能跑吗
regression_pass	回归测试通过吗
fix_rounds	修了几轮
time_to_fix_min	修复用时

中期结果：

- A1 9 条 bug repair 全通过。
- A2 9 条 bug repair 全通过。
- 总计 18 条。

边界：

这说明流程和表单可用，
但样本太小，不能说大样本稳健性已经被证明。

9. CLI 原型 `run_research_case.py` 是什么

这个文件很重要，但它不是重新跑实验的脚本。

它的作用是：

把已有的题目、task card、prompt、run 文档和 metrics 摘要串起来展示。

你可以把它理解成“研究资产导航器”。

9.1 你输入什么

常用命令：

```
python3 run_research_case.py --demo advisor
python3 run_research_case.py --list
python3 run_research_case.py --case A1 --mode plain_guidance --track baseline
python3 run_research_case.py --case A2 --mode coe_guided --track bug-repair
python3 run_research_case.py --case A2 --mode direct_answer --track expanded-scope
```

注意：当前 shell 里 `python` 不存在，所以实际应使用 `python3`。PPT 里写 `python` 是一般写法，现场演示可以改口说“本机用 `python3`”。

9.2 它输出什么

例如 `--demo advisor` 会输出：

1. 阶段判断。
2. A2 输入题目。
3. A2 的结构化表示。
4. 当前 prompt 模板。
5. 代表 run 文档。
6. metrics 摘要。
7. 三种 mode 对比。
8. 推荐继续查看的材料。

9.3 `run_research_case.py` 关键结构

CATALOG

这是全文件最重要的数据结构。

它记录：

- A1 / A2 是什么案例。
- 每个案例有哪些 task card。
- 每个 track 对应哪些 prompt。
- 每个 mode 对应哪些 run_id。
- 每个 track 看哪些 metrics 文件。

你可以把 CATALOG 理解成：

研究资产索引表

build_selection(case_id, mode, track)

作用：根据你输入的案例、模式、轨道，把相关材料全部找出来。

输入：

```
case_id = A1 或 A2
mode = direct_answer / plain_guidance / coe_guided
track = baseline / bug-repair / expanded-scope / fresh-generation
```

输出：

```
task cards
prompt template
prompt used
protocol files
run docs
metrics summary
```

read_csv_rows / rows_by_run_id

作用：读取 metrics CSV，然后只保留当前 run_id 对应的行。

summarize_guidance_eval

作用：总结过程质量。

它会算：

- 有多少行。
- runnable 多少通过。
- correct 多少通过。
- self_check 多少满分。
- 平均首次跑通时间。

summarize_a2_recovery_perf

作用：总结 A2 图像恢复质量。

它会算：

- 平均 PSNR。
- 平均 SSIM。
- 平均 RSE。

- output_ok 数量。
- 图像和 corruption 列表。

render_demo_text

作用：把前面查到的材料组织成导师演示文本。

所以这个 CLI 的汇报价值是：

让导师看到仓库不是散乱文件，而是可以用单一入口追溯的研究链路。

10. PPT 生成脚本做什么

中期 PPT 文件是：

report/大研中期汇报.pptx

生成脚本是：

report/build/build_midterm_pptx.py

它用 python-pptx 生成幻灯片。

10.1 里面的工具函数

函数	作用
add_blank_slide	新建空白页并设置背景
add_title_bar	添加顶部标题栏
add_textbox	添加普通文本框
add_bullets	添加项目符号
add_card	添加卡片
add_code_block	添加代码块样式
add_table	添加表格
add_image	插入图片
add_callout	添加强调框
set_notes	写入演讲备注

10.2 slide 函数

每个 `slide_xxx(prs)` 函数对应一页 PPT，比如：

- `slide_scope_boundary`：题目与阶段边界。
- `slide_phase_timeline`：阶段定位。
- `slide_a1_showcase`：A1 展示。
- `slide_a2_metrics`：A2 指标。
- `slide_a2_critical_finding`：A2 指标一致的边界说明。
- `slide_conclusion`：结论与下一步。

10.3 输入输出

输入：

```
report 下的总结文档
metrics 下的结果表
outputs 下的代表图片
代码片段字符串
```

输出：

```
report/大研中期汇报.pptx
```

所以 PPT 不是孤立材料，而是从代码、输出、CSV 和总结里抽取出来的展示层。

11. 你每一步输入了什么、输出了什么

第一步：收束题目

输入：

```
大模型辅助数学建模教学
早期 Timeline
课程作业材料
```

输出：

```
A1 Seam Carving
A2 RSLT inpainting
```

第二步：写 Task Card

输入：

原始作业要求
你对算法的理解
常见错误
评测需求

输出:

task_cards/A1_seam_carving_taskcard_v1.md
task_cards/A2_rslt_inpainting_taskcard_v1.md

第三步：写 Prompt 模板

输入:

题目要求
Task Card
评测协议
direct / plain / coe 三种条件设计

输出:

prompts/a1/*.md
prompts/a2/*.md

第四步：模型生成代码

输入:

prompt_used.md

输出:

solutions/.../generated/run_xxx/src/*.py
model_raw_response.md

第五步：本地运行评测

A1 输入:

bing1.png
original.png
seam_carve_image

A1 输出:

```
seam_width.png  
seam_height.png  
compare_grid.png  
runtime evidence
```

A2 输入:

```
lena / barbara  
random_pixel@50% / text@50%  
rslt_inpainting
```

A2 输出:

```
compare.png  
summary.json  
eval_summary.json  
PSNR / SSIM / RSE / runtime
```

第六步：写 run 文档

输入:

```
实际运行结果  
错误情况  
修复轮次  
输出路径
```

输出:

```
runs/2026-xx-xx_run_xxx_*.md
```

第七步：回填 metrics

输入:

```
run 文档里的结果  
输出文件是否存在  
指标计算结果
```

输出:

```
metrics/*.csv
```


第八步：写阶段总结和 PPT

输入：

```
run docs
metrics
代表图
协议文档
```

输出：

```
report/*summary*.md
report/大研中期汇报.pptx
```

12. 中期 PPT 每部分讲什么

第一部分：选题与方法

告诉老师：

```
我不是研究“让 AI 直接给答案”，
而是研究“如何把 AI 生成过程变成可检查证据链”。
```

第二部分：案例与框架

告诉老师：

```
我为什么选 A1/A2，
怎么把输入层、过程层、产物层串起来，
怎么做结构化检查和错误分类。
```

第三部分：实验结果

告诉老师：

```
A1 结果：三组都正确，但 plain/coe 自检更完整。
A2 结果：三组最终指标完全一样，不能夸大。
bug repair：18 条都能诊断、修补、回归。
```

第四部分：原型与预试验

告诉老师：

```
我已经有 CLI 原型和 dry run，
S01-S04 四个正式 pilot 任务包已经准备好。
```

第五部分：综合与价值

告诉老师：

这个研究对 USTC 的意义在于：
AI 已经进入作业，
不能只禁用或放任，
需要一种能保留过程证据的作业和评分方式。

13. 你汇报时最应该抓住的主线

你可以这样讲：

我从一个很宽的题目开始，逐步收束到两个可运行、可评分、可复查的主案例。A1 用 Seam Carving 检查动态规划边界、尺寸变化和自检覆盖；A2 用 RSLT 图像修复检查 mask 不变量、低秩分解和数值指标。每个案例都不是只生成一次答案，而是经过 prompt、代码、输出、run 文档、metrics 和总结的证据链。中期最稳的结论是，结构化指导更能提升过程质量和可复查性，但当前还不能说它显著提高最终结果指标，也不能说大样本教学效果已经验证。

14. 老师问“你的代码到底做了什么”时怎么答

可以分三层回答。

第一层：实验组织代码

代表：

```
run_research_case.py  
report/build/build_midterm_pptx.py
```

作用：

组织已有研究资产，生成演示入口和 PPT。

第二层：算法实现代码

代表：

```
A1: seam_carving_generated.py  
A2: a2_generated.py / chapter5_rslt.py
```

作用：

真正完成 Seam Carving 和 RSLT inpainting。

第三层：评测与证据代码

代表：

```
run_protocol_eval.py
metrics/*.csv
runs/*.md
```

作用：

把代码放进固定协议运行，保存输出，并把结果回填成可比较数据。

15. 你最不能说错的边界

一定不要说：

结构化指导显著提高 A2 图像恢复质量。

因为 A2 三组指标完全一样。

应该说：

在 A2 固定协议下，三组最终图像恢复指标一致。
因此 A2 支持的结论是：结构化指导主要提升过程说明、自检覆盖和可复查性，而不是提升最终图像质量。

也不要说：

已经证明了大样本教学效果。

应该说：

中期前完成的是案例闭环和采集准备。正式受试者数据还没开始，后续 S01-S04 pilot 才会补这一部分。

16. 用一句话复盘每个大步骤的衔接

题目收束

让研究从泛泛的 AI 教育落到可执行案例。

Task Card

把题目变成可检查的输入输出、约束和算法步骤。

Prompt 模板

把 direct / plain / coe 三种指导条件固定下来。

生成代码

产生可以运行和评测的 artifact。

评测脚本

在相同协议下跑不同生成结果。

outputs

保存图像和 JSON，形成肉眼可查的结果证据。

run 文档

记录每次实验发生了什么。

metrics CSV

把文字记录变成可横向比较的数据。

summary / PPT

把数据收束成中期能讲的研究结论。

pilot task pack

把研究者自测推进到真实参与者试用。

17. 中期为止最简洁的结论

如果只允许你说 30 秒，可以这样说：

中期前我已经把 A1 Seam Carving 和 A2 RSLT 图像修复两个主案例跑成闭环。每个案例都有题目要求、Task Card、三类 prompt、生成代码、运行输出、run 文档和 metrics 表。A1 显示三组提示最终都能做对，但 plain / CoE 的自检覆盖更完整；A2 显示三组最终图像指标完全一致，所以不能说结构化提示提升最终图像质量，只能说它提升过程质量和可复查性。18 条 bug repair 说明当前错误分类、诊断、补丁和回归验证流程能落地。下一步是用 S01-S04 四个正式 pilot session 验证这套流程在真实参与者身上是否可用。